Music Trends: A Semantic Web Approach to Music Data Analysis

Semantic Web - Practical Assignment 2

Ângela Ribeiro (109061), António Alberto (114622), Hugo Castro (113889)
angelammaribeiro@ua.pt, antonio.alberto@ua.pt, hugocastro@ua.pt
University of Aveiro
Master's in Computer Engineering
Academic Year 2025/2026

***Abstract*—This report presents the evolution of the Music Trends WS system for Practical Assignment 2. The project extends the original music chart analysis application with a richer Semantic Web layer, including an RDFS and OWL ontology for songs, artists, genres, albums, charts, countries, and chart entries. SPIN rules are used to materialize inferred classifications, such as long-tail songs, while SPARQL queries support new analytical operations over the graph. The system also integrates external knowledge from Wikidata and DBpedia to enrich artists with linked semantic data, enabling country-based insights, complementary artist descriptions, and improved artist detail pages. Finally, the Django interface was extended with dashboard components that expose these semantic enrichments and inferred results to users.**

***Index Terms*—RDF, RDFS, OWL, SPIN, SPARQL, GraphDB, Wikidata, DBpedia, Django, Semantic Web, Music Trends.**

I. Introduction to the Theme

Music Trends WS analyzes chart-driven music data (songs, artists, genres, albums, countries, and chart entries) in a domain that is rich but heterogeneous. Conventional tabular exploration is limited for this type of relational knowledge, motivating a Semantic Web approach [1].

TP1 provided the baseline application and initial exploration workflows. TP2 extends that baseline with an explicit semantic layer: an RDFS/OWL ontology, SPIN inference rules, richer SPARQL operations, Wikidata/DBpedia enrichment, RDFa/microformat publication strategy, and UI features that expose inferred and enriched results. The key objectives are to formalize the domain, derive new classifications, integrate external linked data, and present these outcomes in dashboard and detail views.

Technically, the system combines Django, GraphDB, an RDF generation/loading pipeline, SPARQL query/update operations, and enrichment scripts.

II. Ontology Definition (RDFS and OWL)

A. Scope and Design Goals

The ontology, defined in `normalizing_data/ontology.ttl`, provides the semantic layer for chart-based music analytics. Its goal is to represent music entities and their relations as reusable graph objects, enabling richer queries, clearer semantics, and compatibility with inference and external linked data.

B. Core Classes

The central classes are `type:Song`, `type:Artist`, `type:ChartEntry`, `type:Chart`, `type:Genre`, `type:Album`, and `type:Country`. Among these, the key modeling decision is `type:ChartEntry` as a separate temporal observation entity, where rank, weeks, and date are attached to an occurrence in a chart, not to a song as permanent attributes.

Supporting conceptual classes were also introduced for future analytics and domain extensibility, including `type:TimePeriod`, `type:Decade`, `type:Season`, `type:RecordLabel`, `type:SoloArtist`, and `type:BandOrGroup`.

C. Object Properties and Relationships

The ontology models domain structure through explicit relations such as:

- Song → performer → Artist;
- ChartEntry → song → Song;
- ChartEntry → inChart → Chart;
- Artist → originCountry → Country;
- Song → hasGenre → Genre;
- Song → album → Album.

This structure captures relationships directly in the graph, improving traceability and analytical expressiveness.

D. Datatype Properties

Literal attributes are explicitly typed to improve quality and query behavior: `xsd:integer` (e.g., rank, weeks), `xsd:decimal` (audio features and popularity), `xsd:boolean` (explicit), and `xsd:date` (chart/enrichment dates). This avoids ambiguous string-based values and supports reliable filtering, ordering, and comparisons.

E. RDFS and OWL Axioms

The ontology uses practical RDFS/OWL constraints aligned with project needs:

- `rdfs:domain`/`rdfs:range` to define expected subject and object types;
- `rdfs:subClassOf` to organize specialized classes;

- `rdfs:subPropertyOf` to generalize `mainArtist` and `featuredArtist` under `performer`;
- `owl:inverseOf` for bidirectional navigation;
- `owl:equivalentProperty` for backward compatibility;
- `owl:SymmetricProperty` for collaboration relations.

For example, `mainArtist` and `featuredArtist` are modeled as subproperties of `performer`, so queries over `performer` can retrieve both primary and featured artists without additional query logic.

F. Annotation, Validation, and Design Choices

Classes and properties include `rdfs:label` and `rdfs:comment` annotations for readability and maintainability. The ontology was parsed successfully in the RDF pipeline and loaded in Protégé to inspect the class/property hierarchy and verify that the vocabulary is usable in ontology tools.

Key design choices include modeling countries as URI resources instead of plain literals, preserving compatibility properties for legacy data, and maintaining `ChartEntry` as a first-class entity for temporal chart semantics. Modeling countries as resources enables Wikidata-based enrichment to support country-level analytics while still preserving `originCountryLabel` as a fallback for compatibility. This ontology serves as the semantic foundation for the SPIN rule set presented in the next section.

III. Inference Set (SPIN)

A. Motivation for Rule-Based Inference

RDFS/OWL axioms provide schema-level semantics (class hierarchies, subproperties, inverses, equivalences), but they are not sufficient for conditions that require filters, thresholds, date comparisons, and aggregation. In this project, several relevant analytics concepts depend on numeric constraints over chart data. Therefore, we implemented rule-based inference with SPIN/SPARQL to materialize derived facts [3], [2].

B. Rule Implementation Workflow

Rules are explicitly implemented in the independent module `normalizing_data/spin_rules.py`. Each rule defines a `CONSTRUCT/INSERT WHERE` pattern with a clear semantic target. The workflow is:

1) ontology defines the vocabulary (classes/properties);
2) SPIN rules infer new classes and relations;
3) inferred triples are materialized in GraphDB;
4) SPARQL queries consume inferred facts;
5) Django views/templates expose them in the UI.

The same module can also export a SPIN RDF representation (`normalizing_data/spin_rules.ttl`) for inspection and portability.

C. Implemented Rule Set

To improve readability, rules are grouped by purpose instead of only listed by name.

The first five are *classification rules* (new semantic classes), while the last two are *relation-creation rules* (new graph links) (see Table I).

D. Example: LongTailSongRule

`LongTailSongRule` is the most representative non-trivial rule. It groups chart entries by song, computes each song's `MAX(pred:weeks)`, and classifies the song as `type:LongTailSong` when the maximum is at least 20 weeks.

This rule captures persistence in charts, distinguishing long-lasting songs from songs with only short-term peak performance. It is analytically relevant for trend interpretation and cannot be naturally captured by ontology axioms alone because it requires aggregation and threshold evaluation.

E. Execution, Validation, and Impact

Materialization is executed with:

```
python normalizing_data/spin_rules.py
--apply
```

After materialization, inferred triples become directly queryable in GraphDB. Validation is performed through SPARQL count/ranking queries (for example, counting `type:LongTailSong` instances and ranking songs by longevity) and by observing derived indicators in dashboard components. In this way, inference has visible application-level impact, not only schema-level value.

F. Limitations

The adopted approach is materialized inference: when source data changes, rules must be re-applied to refresh derived triples. This is a conscious trade-off between operational simplicity and immediacy of query-time reasoning.

Table I
IMPLEMENTED SPIN RULES AND INFERRED SEMANTICS

Rule	Input pattern	Inferred result	Purpose
`ChartedSongRule`	Song appears in at least one chart entry	`ChartedSong`	Identifies songs that entered a chart
`HitSongRule`	Chart entry with `rank <= 10`	`HitSong`	Identifies top-performing songs
`LongTailSongRule`	`MAX(weeks) >= 20`	`LongTailSong`	Identifies songs with chart longevity
`HitArtistRule`	Artist performs a `HitSong`	`HitArtist`	Identifies artists with hit songs
`TrendingArtistRule`	Artist has top-10 song after 2024-01-01	`TrendingArtist`	Highlights recent high-performing artists
`AppearsInChartRule`	Artist performs song appearing in chart	`appearsInChart`	Simplifies artist/chart queries
`CollaboratedWithRule`	Two artists perform the same song	`collaboratedWith`	Creates artist collaboration links

IV. NEW DATA OPERATIONS (SPARQL)

A. Purpose of SPARQL Operations

SPARQL turns the RDF graph into analytical insights. In this architecture, the ontology provides structure, SPIN rules materialize derived triples, and SPARQL retrieves, aggregates, filters, and updates those triples for both analysis and application features [2].

B. SELECT Queries for Analytical Exploration

Instead of organizing queries by file or endpoint, operations were designed by analytical purpose:

- **Semantic coverage:** counts of inferred classes such as `type:LongTailSong`, `type:HitSong`, and `type:HitArtist`;
- **Ranking and trend analysis:** longest-charting songs and artists with the highest number of hits;
- **Dimensional analysis:** distributions and comparisons by country, genre, chart, album, and time dimensions;
- **Semantic navigation:** exploration of inferred relations such as collaborations and artist–chart links.

The recurring pattern is: analytical question → SPARQL aggregation/filter pattern → interpretable result. For example, long-tail analysis queries instances of `type:LongTailSong` and ranks them by `MAX(pred:weeks)`, allowing the dashboard to highlight songs with the longest chart presence.

C. UPDATE and Materialization Operations

The project performs not only query-time reads but also update-time materialization of inferred knowledge. Rules defined in `normalizing_data/spin_rules.py` are executed as SPARQL-based materialization rules, creating persistent triples in GraphDB:

```
python normalizing_data/spin_rules.py
--apply
```

After this step, classes such as `type:LongTailSong` are directly available to subsequent SELECT queries, avoiding repeated recomputation of rule conditions in UI-oriented operations.

D. Validation and Application Impact

Query results were validated through count checks, sample ranking queries, and cross-verification in dashboard components. This creates a complete chain from inference to user-visible analytics: inferred triples become queryable metrics, and those metrics are presented in insights and dashboard views.

E. Limitations

Two practical limitations were identified. First, because inference is materialized, rules must be re-applied whenever the base graph changes. Second, aggregation-heavy queries (e.g., rankings and grouped distributions) are computationally heavier than direct lookup queries, requiring careful use in interactive views.

V. USE AND INTEGRATION OF WIKIDATA AND DBPEDIA DATA

A. Entity Linking Strategy

Integration is implemented in `normalizing_data/enrich_artists.py` using `SPARQLWrapper`. Local artists are matched to external entities through name-based lookup with validation and conservative fallback behavior. When a match is accepted, traceability is explicitly represented with `owl:sameAs`, allowing the local node to remain connected to Wikidata and/or DBpedia resources [4], [5].

B. Data Retrieved from Wikidata and DBpedia

The two sources are used in a complementary way:

- **Wikidata (structured facts):** origin country, external genres, image, dates, website, and MusicBrainz identifiers;
- **DBpedia (descriptive enrichment):** thumbnail, birth place, and abstract text when available.

This separation avoids treating both knowledge bases as interchangeable and better reflects their practical strengths in the enrichment pipeline.

C. Integration into the Local RDF Graph

External data is added to the RDF graph (not only consumed in the UI). A typical integrated pattern is:

```
ex:artist_x owl:sameAs wd:Q... ;
pred:originCountry wd:Q... .
```

```
wd:Q... a type:Country ; rdfs:label
"..." .
```

This pattern illustrates URI-based modeling and is consistent with the country-as-resource design adopted in the ontology.

D. Impact on SPARQL Queries and UI

The enriched graph enables country-level analytics, broader artist metadata, and better explainability in detail pages. In practice, artist views display external metadata and descriptive text; DBpedia abstract content is supported by the model and shown when present in the current enrichment snapshot, with fallback to Wikidata/local description values otherwise.

E. Validation, Coverage, and Limitations

Coverage was validated from generated enrichment triples (`normalizing_data/artists_external.ttl`). In the current snapshot, `owl:sameAs` links are present for 1902 artist subjects, origin-country resource links for 1372 artists, and description literals for 1890 artists. These counts confirm broad but non-uniform enrichment coverage.

Main limitations are expected in open linked data integration: variable field availability across entities, ambiguity in name matching, and uneven DBpedia abstract coverage. Consequently, the system uses fallback logic and avoids assuming complete external metadata for every artist.

VI. Publishing Semantic Data through RDFa and Microformats

A. Purpose of Semantic Publishing

Semantic publishing with RDFa and microformats extends the system from internal SPARQL consumption to a data model that is also readable and reusable directly in application HTML. The objective is to expose semantics in rendered pages that can be consumed by external agents.

B. RDFa and Microformat Strategy

RDFa is used as the main expressive layer, mapping page elements to semantic entities and properties through attributes such as `typeof`, `property`, and `resource`. Microformats are used as a lighter complement for simpler consumers and human-centered metadata extraction [6], [7].

C. Annotated Pages and Semantic Mapping

We chose to embed RDFa in `artist_detail.html` and Microformats2 in `song_detail.html`. In the artist page, the top-level entity is typed as `schema:Artist` with `owl:sameAs` links to Wikidata, DBpedia, and MusicBrainz, grounding the local URI in external knowledge bases. Artist properties include name, genre, description, founding date, and country of origin. Each song in the tracklist is annotated as a `schema:MusicRecording` with audio features (energy, danceability, popularity) typed as `xsd:decimal`, and linked back to the artist via `byArtist`. Collaborators are expressed using a custom `collab` predicate, each typed as `schema:Artist`.

In the song page, Microformats2 is used: the song is marked up as an `h-entry` with `p-name`, `u-url`, and `p-category` for the song title, URI, and genre respectively. The main artist and any featured artists are embedded as nested `h-card` objects under `p-author`, capturing name and profile URL. This naturally supports multi-artist tracks, as demonstrated by *Die With A Smile* where both Lady Gaga and Bruno Mars appear as co-authors.

A compact RDFa-style example is:

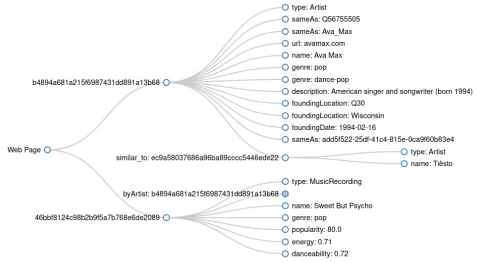

Figure 1. RDFa graph output for Ava Max artist page

A compact Microformats example is:


```
{
    "items": [
        {
            "type": ["h-entry"],
            "properties": {
                "name": ["Die With A Smile"],
                "url":  ["http://music.org/song/4d307eb0a98e58d0d1dc2492f76638de"],
                 "category" : ["pop"],
                "author": [
                    {
                        "type": ["h-card"],
                        "properties": {
                            "name": ["Lady Gaga"],
                            "url":  ["http://music.org/artist/f4a1462da44612fd2c5e1adc4fb7842c"]
                        }
                    },
                    {
                        "type": ["h-card"],
                        "properties": {
                            "name": ["Bruno Mars"],
                            "url":  ["http://music.org/artist/ce178f34069e29a506c407052db01ca7"]
                        }
                    }
                ]
            }
        }
    ]
}
```


Figure 2. Microformats parse for song "Die with a Smile"

VII. Application Features (UI), Not Present in TP1 or Complementary

A. Purpose of the UI Evolution

The TP2 user interface operationalizes the semantic gains of the system by transforming ontology definitions, inferred classes, and linked external data into concrete analytical features for end users. In TP1, the interface mainly exposed base dataset exploration; in TP2, it exposes derived and enriched knowledge.

B. Semantic Dashboard Additions

Dashboard components were extended to present semantic analytics rather than only raw counts. These additions combine ontology relations, inferred classes, and external enrichment signals in user-facing indicators.

C. Country-Based Insights and Drill-Down

Country-oriented views are supported by the `pred:originCountry` relation (enriched from external sources and modeled as URI resources). SPARQL aggregations over country resources power views such as top countries by charted songs and country drill-downs with top artists/songs. This turns linked-data enrichment into directly visible geographic analysis.

D. Inferred Long-Tail Song Metrics

Long-tail indicators are a direct UI consequence of semantic inference. `type:LongTailSong` is inferred by SPIN (not stored in the original dataset), materialized in GraphDB, and then consumed by SPARQL queries used by dashboard cards and rankings. This provides a complete pipeline from rule-based inference to user-visible trend metrics.

E. Enriched Artist Detail Pages

Artist detail pages now combine internal chart data with external semantic context, including origin country, external links, media metadata, and description text. DBpedia abstract-style content is shown only when present in the current enrichment snapshot; fallback values from other sources are used when external fields are missing.

F. Enriched Song Detail Pages

Song detail pages now include lyrics fetched from the LRCLIB.net API, displayed on demand via a modal [8].

G. Collaboration and Semantic Navigation

The interface also supports semantic navigation over artist relations, including collaboration links derived from shared performances. This enables exploration patterns such as artist $\rightarrow$ `collaboratedWith` $\rightarrow$ artist, complementing song/chart views with graph-style relationship browsing.

VIII. Conclusions

TP2 transforms the system from base dataset exploration into a semantic music analytics platform that combines ontology, rule-based inference, external linked data, and UI-visible analytical features.

The main contributions are:

- an RDFS/OWL ontology for songs, artists, charts, countries, and chart-entry semantics;
- SPIN materialization rules for derived classes and relations (e.g., `type:LongTailSong`);
- SPARQL operations organized for analytical exploration, ranking, and semantic navigation;
- Wikidata/DBpedia enrichment with explicit `owl:sameAs` links and external semantic context;
- UI extensions that expose semantic insights through country analytics, inferred long-tail metrics, enriched artist pages, and relation browsing.

Some limitations remain relevant: inferred data must be rematerialized after base graph updates and external enrichment coverage depends on source quality and availability.

Future work would focus on extending semantic publishing coverage, improving validation of external links and mappings, and adding finer-grained temporal inference and enrichment strategies.

TP2 demonstrates that combining ontological modeling, rule-based inference, and linked external data improves not only semantic representation of the music domain, but also the analytical functionality available to end users.

IX. Configuration to Run the Application

A. Prerequisites

- Python 3.10+;
- GraphDB running and reachable (default: http://localhost:7200);
- project dependencies installable from `requirements.txt`.

Internet access is required only when re-running external enrichment from Wikidata/DBpedia; pre-generated enrichment artifacts can be loaded without repeating that step.

B. Recommended Full Execution

The recommended end-to-end path is:

```
./setup.sh full
```

This command installs dependencies, generates RDF, enriches artists, loads/reloads GraphDB content, applies SPIN materialization, and starts the Django server.

C. Manual Execution Steps

Manual commands on Windows:

```
1) python -m venv venv
2) venv\Scripts\activate
3) pip install -r requirements.txt
4) cd normalizing_data
5) python create_rdf.py
6) python enrich_artists.py --patient
7) python load_ttl_to_graphdb.py
8) cd ..\django\music_trends
9) python manage.py migrate
10) python manage.py runserver
```

Manual commands on macOS/Linux:

```
1) python3 -m venv venv
2) source venv/bin/activate
3) pip install -r requirements.txt
4) cd normalizing_data
5) python3 create_rdf.py
6) python3 enrich_artists.py --patient
7) python3 load_ttl_to_graphdb.py
8) cd ../django/music_trends
9) python3 manage.py migrate
10) python3 manage.py runserver
```

`load_ttl_to_graphdb.py` already applies SPIN rules after loading.

D. Delivery Files

The file containing only the facts, without typology to be imported into GraphDB: `facts_only.ttl`

The file containing only the ontology to be imported into GraphDB: `ontology.ttl`

The file integrating the ontology with the facts, so it can be tested and validated in Protégé: `integration_protege.ttl`

The SPIN inference rules: `spin_rules.py` / `spin_rules.ttl`

Protégé validation workflow is documented in `protege_validation.md`.

REFERENCES

[1] T. Berners-Lee, J. Hendler, and O. Lassila, "The Semantic Web," *Scientific American*, vol. 284, no. 5, pp. 34–43, 2001.

[2] S. Harris and A. Seaborne, "SPARQL 1.1 Query Language," W3C Recommendation, Mar. 2013. [Online]. Available: https://www.w3.org/TR/sparql11-query/

[3] H. Knublauch, J. A. Hendler, and K. Idehen, "SPIN - SPARQL Inferencing Notation," W3C Member Submission, Feb. 22, 2011. [Online]. Available: https://www.w3.org/Submission/SPIN-overview/"

[4] D. Vrandečić and M. Krötzsch, "Wikidata: A Free Collaborative Knowledgebase," *Communications of the ACM*, vol. 57, no. 10, pp. 78–85, 2014.

[5] S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, and Z. Ives, "DBpedia: A Nucleus for a Web of Open Data," in *Proc. ISWC/ASWC*, 2007, pp. 722–735.

[6] B. Adida, M. Birbeck, S. McCarron, and I. Herma, "RDFa Core 1.1—Third Edition," W3C Recommendation, Mar. 2015. [Online]. Available: https://www.w3.org/TR/rdfa-core/

[7] Microformats Community, "Microformats2," Draft Specification. [Online]. Available: https://microformats.org/wiki/microformats2

[8] LRCLIB.net, “LRCLIB.net” [Online]. Available: https://lrclib.net/docs [Accessed: 2025]